AR / VR (IS372IA) – Exam Notes

CIE-1 & Quiz-1 · Unit I (complete) + Unit II (Programming with Unity) · built only from the uploaded course notes

How to read these notes

- **Green box** = definition to write word-for-word in exams. **Yellow box** = exam tip / likely question. **Purple box** = a diagram/image in your PDFs (I've given the file name and page so you can open it). **Red box** = a mistake or inconsistency in the notes.
- Figure references use the format `pdfname_pg_N` (e.g. `UNIT1_pg_13` = page 13 of UNIT1.pdf).

UNIT I – Introduction & Bird's-eye view

Syllabus: VR, AR, MR, applications · Hardware, Software, Human physiology & perception, History of VR and AR

1. Virtual Reality (VR)

Sources: UNIT1, VRAndApplications, VRAR-Introduction_PPT_1_, virtual_reality_notes_docx-1, Hardware_and_software_VRAR

1.1 Definitions

> **Definition 1 (short):** An artificial world consisting of images and sound created by a computer and affected by the actions of the person experiencing it. *(UNIT1 pg 2)*

> **Definition 2 (formal):** Virtual Reality is the process of inducing **targeted behavior** in an **organism** using **artificial sensory stimulation**, while the organism has **little or no awareness** of the interference. *(VRAndApplications pg 2)*

> **Definition 3:** A three-dimensional artificial environment created using computer hardware and software and presented so that it appears and feels like a real environment. The user enters it using special gloves, goggles, earphones etc., which contain sensors to detect the required signals. *(virtual_reality_notes pg 1)*

> **Definition 4:** A computerized simulation of natural or imaginary reality; the user is fully or partially immersed. *(virtual_reality_notes pg 3)*

- VR is **not just a headset or a game** – it is a system that replaces or enhances sensory information so the brain accepts a virtual world as real. *(VRAndApplications pg 2)*
- VR shifts a subject into a different environment **without physically moving** them: inputs to the sensory organs are manipulated so the perceived environment is the Virtual Environment (VE), not the physical one. The manipulation is controlled by a **computer model** of the VE. *(virtual_reality_notes pg 4)*
- VR can be text-based or graphics-based. *(virtual_reality_notes pg 1)*

1.2 How VR works & its characteristics

- The person is **immersed in a digital 3D environment** instead of watching a display. *(UNIT1 pg 3; VRAR-Introduction pg 6)*

- Primary goal: simulate **vision**; it also aims to simulate real presence through **hearing and touch**.
- Information is right in front of the eyes, so it **eliminates any interaction with the real world**.

VR characteristic	Meaning
Totally immersive environment	User is completely inside the virtual world; real world blocked out
Senses under control of system	What the user sees/hears/feels is generated by the system
Needs a mechanism to feed the virtual world to the user	Display + tracking + I/O hardware (see Hardware)

(UNIT1 pg 5; VRAR-Introduction pg 8)

> **Figure refs:** VR vs AR illustration – `VRAR-Introduction_PPT_1__pg_7` ("Digital environments that shut out the real world"); stereo split-screen VR view – `UNIT1_pg_3` .

1.3 Four key elements of VR

(VRAndApplications pg 3–7) · figure: `VRAndApplications_pg_3` (four-segment wheel)

Element	Explanation	Examples / points
Targeted behavior	The experience **designed by the creator**; intentional, not random	Walking or flying, learning or training, social interaction
Artificial sensory stimulation	VR **replaces or enhances** natural sensory input using technology; at least one sense is partially controlled artificially	Vision, hearing, touch, motion & balance
Unawareness	Organism is unaware of the interference	Acceptance of the virtual environment, feeling of "being there", perceptual illusion – like an optical illusion
Organism	The one experiencing VR	Human user or animal (rodents, insects, primates); widely used in neuroscience experiments, so VR is not limited to humans

Is the brain really fooled? *(VRAndApplications pg 8)*

- **Place cells** activate in virtual spaces; **grid cells** encode virtual locations.
- The brain maps virtual environments like real ones → VR can **partially** fool the brain.

1.4 Immersion

(virtual_reality_notes pg 4–5, 2–3)

- Immersion is a **key issue** in VR – the user becomes part of the simulated world, rather than the simulated world being a feature of the user's world.
- The first "immersive VR systems" were **flight simulators** (a mix of real hardware and virtual imagery).
- **Necessary condition** (Ellis' notion of a VE): a VE maintained in at least one sensory modality, typically visual. E.g. a head-mounted display with **wide field of view and at least head tracking** is essential.
- **Degree of immersion increases** with: more and consistent sensory modalities, greater body tracking, richer body representations, **decreased lag** between body movement and sensory change.
- **Astheimer:** immersion = the feeling of the VR user that the VE is real. Analogous to Turing's definition of AI: if the user cannot tell which reality is "real" and which is "virtual", the computer-generated one is immersive.
- **Full immersion** = using a machine to shield oneself from the real world. **Partial immersion** = the person can manipulate a VR environment without being locked away in a machine. Full immersion is not compulsory – *Second Life* and the Nintendo Wii remote are VR-based products.

1.5 The three I's of VR (VR triangle)

Immersion · Interaction · Imagination (written I^3). *(UNIT1 pg 7)*

> **Figure ref:** `UNIT1_pg_7` – "Virtual Reality Triangle" with Immersion, Interaction and Imagination at the three corners.

1.6 The five classic components of a VR system

VR Engine · I/O Devices · Software & Databases · User · Task. The VR engine and I/O devices exchange data both ways; the VR engine is linked to software & databases; the I/O devices connect to the user; the user performs the task. *(UNIT1 pg 8)*

> **Figure ref:** `UNIT1_pg_8` – "VR System Architecture" block diagram.

1.7 Interactivity: open-loop vs closed-loop

	Open-loop	**Closed-loop**
Behaviour	Sensory output **without responding** to user actions	**Continuously updates** sensory output based on user actions (feedback)
Examples	Watching a 360° VR video	VR games, flight or driving simulators, training simulations
Diagram	System → User	System → User → Feedback → System

(VRAndApplications pg 10) · figure: `VRAndApplications_pg_10`

1.8 Synthetic vs captured virtual worlds

- **Synthetic:** fully computer-generated worlds. **Captured:** real environments scanned and reconstructed.
- **Avatars** represent users in shared VR spaces. *(VRAndApplications pg 11)*

1.9 Types of VR: immersive, non-immersive, shared

Characteristics of immersive VR *(virtual_reality_notes pg 11)*

- **Head-referenced viewing** – natural interface for navigating 3D space; allows look-around, walk-around, fly-through.
- **Stereoscopic viewing** – enhances depth perception and sense of space.
- Virtual world presented at **full scale**, relating properly to human size.
- **Realistic interaction** with virtual objects via data glove etc. (manipulation, operation, control).
- Illusion enhanced by **auditory, haptic** and other non-visual technologies.
- **Networked applications** allow shared virtual environments.

Shared virtual environments *(pg 12–13)*

Users at different locations meet in the same virtual world, e.g. one using a BOOM, one a CAVE and one an HMD. All see the same VE from their own viewpoint; each is shown as a virtual human (**avatar**); they can see, communicate with and interact as a team. Figure: `virtual_reality_notes_docx-1_pg_13` (Fig 4.2) and types of VR `virtual_reality_notes_docx-1_pg_12` (Fig 4.1).

Non-immersive VR *(pg 13)*

VR applications that are not fully immersive: mouse-controlled navigation through a 3D environment on a monitor, stereo viewing via stereo glasses, stereo projection systems. Example: Apple **QuickTime VR** – uses photographs to model 3D worlds with pseudo look-around and walk-through on a monitor.

VRML *(pg 13–14)*

- **Virtual Reality Modeling Language** – brings 3D worlds with integrated hyperlinks to the Web (HTML gives "home pages", VRML gives "home spaces").
- Viewed with a browser plug-in under mouse control → not fully immersive, but its syntax/data structure is excellent for modelling functional, interactive 3D worlds that can later move to fully immersive systems.
- VRML 2.0 = international ISO/IEC standard **VRML97**.

VR-related technologies *(pg 14)*

- **Motion trackers** monitor dancers/athletes for later study in immersive VR.
- **Augmented reality** – real environments with superimposed virtual objects.
- **Telepresence** (telemedicine, telerobotics) – immerses a viewer in a real world captured by distant cameras and allows remote manipulation of real objects via robot arms.

1.10 Terminology

Term	Meaning
VR	Fully simulated environment
AR	Digital overlays on the real world
MR	Combination of real and virtual
XR	Umbrella term for all realities

(VRAndApplications pg 9)

1.11 Technology note (Unity & Vuforia)

- Unity is the most widely used VR, AR or MR development platform; the notes state that over 91% of Vuforia and HoloLens experiences are made with Unity. *(UNIT1 pg 6; VRAR-Introduction pg 11)*
- **Vuforia Engine** – software platform for creating AR apps; adds computer-vision functionality so apps can recognise images and objects and interact with spaces in the real world.

2. Augmented Reality (AR)

Sources: ARandApplications, VRAR-Introduction_PPT_1_, Hardware_and_software_VRAR, Human_Physiology...

2.1 Definitions

> **AR:** A combination of a real scene viewed by a user and a virtual scene generated by a computer that augments the scene with additional information. *(VRAR-Introduction pg 3)*

> **Basic idea:** to superimpose graphics, audio and other sense enhancements over a real-world environment in real time. The graphics change to accommodate the user's hand or head movements. *(VRAR-Introduction pg 4)*

> **Short:** AR adds digital objects to the real world; the user still sees the real environment and virtual objects appear on top of real ones. Interaction happens via mobile phones, tablets, AR glasses. *(Hardware_and_software pg 4)*

2.2 Why AR? (introduction, limitations, motivation)

- Advances in computer graphics made virtual content visually realistic, **but** virtual environments are detached from the physical world; most human activity and interest is in the real world → need to connect digital information directly with the physical world. AR addresses this by enhancing real-world perception. *(ARandApplications pg 1, 4)*

Limitations of traditional digital interfaces *(ARandApplications pg 2)*

- Information on smartphones/computers is disconnected from physical surroundings.
- Users must **mentally associate** digital information with real-world objects → indirect interaction **increases cognitive effort**.
- GPS and barcode scanners need explicit user actions and are coarse-grained and context-limited.

Motivation – Cognitive Load Theory *(ARandApplications pg 3)*

Load type	Meaning
Intrinsic load	Complexity of the new information
Germane load	Linking new information with current information
Extraneous load	Unnecessary and distracting information

Information is accessed through screens separate from the environment → users map it mentally to physical objects → higher cognitive load. **AR aims to create automatic, direct and intuitive links between the real world and digital data.**

> **Figure refs:** AR architecture (User ↔ Device with virtual content ↔ Real-world entity; "interaction" and "tracking" arrows) – `ARandApplications_pg_1` ; cognitive load diagram – `ARandApplications_pg_3` ; real world vs AR overlays on a shop street – `ARandApplications_pg_4` .

2.3 Characteristics of AR (Azuma)

> **An AR system must:** (1) combine real and virtual elements, (2) be interactive in real time, (3) be registered in three dimensions.

Additional: accurate spatial alignment; continuous user–system feedback loop; supports visual, audio and haptic augmentation. *(ARandApplications pg 6)*

> **Figure ref:** `ARandApplications_pg_6` – optical see-through AR schematic: scene generator, head tracker (head locations), monitors (graphic images), optical combiners, real world.

2.4 AR vs VR vs MR

Feature	Virtual Reality	Augmented Reality
Environment	Fully computer-generated / virtual	Real + virtual (enhances the real world)
Real world	Blocked / replaced	Visible; awareness of surroundings maintained
Immersion	Complete	Partial
Devices	VR headset	Phone / tablet / AR glasses
Senses	Under control of the system	User keeps a sense of presence in the real world
Needs	Mechanism to feed virtual world to user	Mechanism to combine virtual and real worlds
Suited for	Replacing surroundings (training, games)	Real-world task support

(ARandApplications pg 5; VRAR-Introduction pg 8; Hardware_and_software pg 5, 25; Human_Physiology pg 2)

Merged (mixed) reality: real and virtual are **intertwined**; interaction with and manipulation of both physical and virtual environment. *(ARandApplications pg 5)*

> **Figure refs:** three-column VR / AR / Merged reality comparison – `ARandApplications_pg_5` ; VR vs AR images – `VRAR-Introduction_PPT_1__pg_7` ; VR vs AR illustration – `Human_Physiology_and_perception_of_VRAR_pg_2` .

2.5 Types of AR devices *(VRAR-Introduction pg 12–15)*

Type	Description
1. Heads-up display (HUD)	Any **transparent display** that presents data without requiring users to look away from their usual viewpoint
2. Holographic displays	Extract people/objects from the real world **in real time** and immerse them into a virtual environment where they interact naturally with digital elements
3. Smart glasses	Wearable **transparent** device that generates AR content in the user's viewpoint; the user sees surroundings as with ordinary glasses
4. Handheld	Popular; **basic hardware requirement**; **touch-screen interaction** is a common method

> **Figure refs:** device photos – `VRAR-Introduction_PPT_1__pg_12` to `_pg_15` ; first-person and third-person controller views – `VRAR-Introduction_PPT_1__pg_16` and `_pg_17` (image-only slides).

2.6 AR tracking techniques *(VRAR-Introduction pg 18–21)*

- **Sensor-based tracking:** (1) optical, (2) magnetic, (3) acoustic, (4) inertial.

- **Vision-based tracking** (camera/image based) – diagram only in the notes: `VRAR-Introduction_PPT_1__pg_21`.

2.7 Advantages and challenges of AR *(ARandApplications pg 14–15)*

Advantages

- Reduces cognitive load
- Context-aware information
- Improves accuracy and task performance
- Enhances learning and training
- Natural, intuitive interaction

Challenges

- Accurate tracking and registration
- Sensor inaccuracies, environmental variability
- Hardware limitations and latency
- Usability and ergonomics
- Scalability and robustness in real-world environments

Conclusion: AR enhances perception of the real world, connects digital information directly with physical context, and is a paradigm shift in user interfaces. *(pg 16)*

3. Mixed Reality (MR)

Source: PPT-Mixed_Reality_and_Applications_1_ (pg 2–10)

> **Mixed Reality (MR):** blends real and virtual worlds seamlessly; digital content **dynamically integrates with and responds to the physical environment**. This is the hallmark that distinguishes MR from VR and AR. *(pg 2–3)*

3.1 Virtuality continuum (Milgram & Kishino, 1994)

- A spectrum from **"pure reality"** (physical world) to **"pure virtuality"** (completely computer-generated, e.g. VR).
- MR occupies the space **between** the extremes and includes **Augmented Reality (AR)** and **Augmented Virtuality (AV)**.
- It gives a model for classifying technologies that merge real and virtual elements.

	VR	AR	MR
What it does	Fully immerses user in a simulated environment	Overlays digital information on the real world	Digital content dynamically integrates with and responds to the physical environment
Example	Playing a game entirely in a digital world	Digital directions overlaid on a real street	Virtual pet interacting with real living-room furniture; digital architectural model placed in a real construction site, reacting to obstacles and lighting

(pg 3)

3.2 How MR blends the worlds – "Physical world integration"

Sensor fusion: combine data from RGB cameras, depth sensors and **IMUs** (Inertial Measurement Units) to get a precise understanding of the physical environment. Real-time **spatial mapping** lets virtual objects react realistically: characters walk behind real furniture, virtual content casts shadows on real surfaces, digital elements are manipulated by hand gestures. *(pg 3)*

3.3 Core technical pillars *(pg 4)*

Pillar	Key points
Environmental perception	**Spatial mapping** (depth sensors, LIDAR, stereo cameras map surfaces, dimensions, layout) → **scene understanding** (recognises/classifies walls, tables, chairs and their semantic relationships) → **world-anchoring**. Gives a 3D reconstruction so virtual objects rest on surfaces or are occluded by real ones
Advanced input systems	**Hand tracking** (precise hand/finger movements → digital input) and **gesture input** (pinch to select, swipe to scroll). More natural than button-based input
Cloud & edge computing	Cloud handles heavy tasks (detailed 3D rendering, AI for scene understanding, multi-user sync). **Edge** nodes near the user handle time-sensitive processing → lower latency. Keeps MR devices lightweight and untethered
High-fidelity rendering	GPU-accelerated holographic graphics; realistic **occlusion** (virtual object hidden behind real one), **spatial sound**, dynamic lighting that adapts to the real environment → presence and immersion

3.4 Spatial mapping & environmental understanding *(pg 5)*

- **Detailed 3D meshes** built by continuous scanning using **time-of-flight (ToF)** sensors (measure distance by emitting and detecting reflected light) and **visual-inertial odometry (VIO)** (camera data + inertial readings to track device movement and build a consistent map). Enables realistic **hologram anchoring**.

- **Scene Understanding SDK:** processes raw mesh data into static high-detail world meshes. **Plane detection** (floors, walls, tabletops), **object recognition** (chairs, doors, windows), **semantic labeling** (assigns meaning).
- **Real-time adaptation:** app responds to changes in the environment (e.g. manufacturing overlay updates when a workbench is moved; navigation cues stay correct when furniture is rearranged).
- **Enhanced realism – Microsoft HoloLens: depth buffering** (correct occlusion) and **surface reconstruction** (virtual content sits on real surfaces rather than floating).

3.5 Input & interaction technologies in MR *(pg 6)*

Technology	Notes
Hand tracking	Articulated hand recognition (computer vision + deep learning), tracks **over 25 joint positions**; manipulate virtual objects without controllers
Eye tracking	**Foveated rendering** – area the user looks at (fovea) rendered in high resolution, periphery in lower detail → saves computation. Also **gaze-based UI control**
Speech & gesture recognition	Neural networks interpret natural-language commands and physical gestures; context-aware understanding of user intent
Extended sensors	**Ultraleap** – infrared cameras, precise 3D hand model, mid-air interaction and **mid-air haptic feedback**. **Kinect** – full-body movement capture for gesture control

3.6 MR software frameworks – MRTK *(pg 7)*

- **Mixed Reality Toolkit (MRTK):** open-source, cross-platform SDK for **Unity**. Provides input systems, spatial awareness, UI controls optimised for MR, simulation tools; modular and extensible.
- **Platform support:** Microsoft HoloLens 2, Meta Quest, Windows Mixed Reality headsets, any **OpenXR**-compatible device.
- **Workflow:** in-editor simulation for rapid prototyping (no need to deploy to a device each time); integrates with Visual Studio and the Windows SDK for building UWP (Universal Windows Platform) MR apps.

3.7 CS challenges & research frontiers *(pg 9)*

- **Real-time perception:** low latency (below 20 ms) and high accuracy, especially in dynamic scenes.
- **Data optimisation:** 3D compression and streaming; **neural rendering** (neural networks generate realistic images from complex 3D data); **mesh simplification** (fewer polygons, same visual fidelity).
- **Security & privacy:** spatial data reveals private spaces; biometrics (eye tracking, hand gestures) are unique identifiers.
- **AI integration:** context-aware interaction, predictive modelling, adaptive user interfaces.

4. Applications of VR, AR and MR

4.1 Applications of VR

Sources: VRAndApplications pg 12–17, Hardware_and_software pg 23, virtual_reality_notes pg 1–2, 15–18

Area	Use
Education	Immersive learning – students explore concepts and environments interactively; better understanding and retention through experiential learning; virtual labs
Healthcare	Medical training, therapy, rehabilitation in controlled virtual environments; reduces risk, improves precision, supports recovery; surgery simulation
Architecture	Architects and clients visualise and experience designs **before construction**; identifies design issues early; improves decision-making
Gaming	Highly immersive, interactive; players feel physically present in the game world
Entertainment	Virtual concerts, movies, theme-park simulations; audience actively experiences rather than passively consumes
Social interaction	Meet in shared virtual spaces using **avatars**; remote collaboration and social presence beyond physical distance
Industry	Safety training

> **Figure refs:** `VRAndApplications_pg_12` (education), `_pg_13` (healthcare), `_pg_14` (architecture), `_pg_15` (gaming), `_pg_16` (entertainment), `_pg_17` (social interaction).

From the VR notes (virtual_reality_notes)

- **Educational potential** (pg 1): data gathering and visualisation; project planning and design; interactive training systems; virtual field trips; experiential learning environments. Also helps non-traditional learners (physically disabled, rehabilitation).
- **Professional applications** (pg 1): robotics, medicine, scientific visualisation, aviation, business, architectural and interior design, city planning, product design, law enforcement, entertainment, visual arts, music, dance.
- **Arts** (pg 2): VR art exhibit at Soho Guggenheim Museum (1993); Banff Center for the Arts; theatre; filmmaking (planning and special effects).
- **Education & training** (pg 16): computerised 3D anatomy atlases for medical students.
- **Medical** (pg 16–17): research curiosity → clinically important area; "Medicine Meets Virtual Reality" yearly meetings.
- **Diagnostics** (pg 17): 3D organ reconstruction from radiological cross-sections; **virtual endoscopy** (virtual colonoscopy/bronchoscopy) – patient comfort and cost effectiveness; virtual colonoscopy as a

screening tool for colon polyps and cancer is in clinical validation.

- **Virtual hospital** (pg 17): medical students practise on a virtual patient, e.g. operating on a dying patient, learning emergency/accident handling.
- **Media** (pg 15–16): Star Trek: TNG (**holodeck**); films *TRON* (first mainstream Hollywood picture on VR) and *The Matrix* (1999); games – *Mage: The Ascension*, *.hack*.

> **Health & safety in VR *(VRAndApplications pg 18)*:** VR can cause discomfort if poorly designed – **VR sickness, fatigue, dizziness or nausea**. Understanding human perception is essential for safe VR.

4.2 Applications of AR

Sources: ARandApplications pg 9–13, VRAR-Introduction pg 23–41, Hardware_and_software pg 23

Area	How AR is used
Industry & construction	Facilities planned with CAD models, but on-site modifications differ. AR overlays CAD models on physical structures → compare **"as-planned" vs "as-is"**; improves accuracy, inspection speed, decision-making
Maintenance & training	Maintenance is complex and documentation-heavy. AR shows **step-by-step instructions directly on equipment**; reduces dependency on manuals; lets less-trained personnel perform tasks correctly; improves training efficiency, safety, consistency
Medical	Integrates X-ray/CT data with live patient view; see inside the body non-invasively; better surgical precision and diagnostics; surgeons visualise bones, muscles, internal organs; **AccuVein** (AR diagnosis)
Navigation & personal information	Directions overlaid on real-world views; less distraction than map-based navigation; **AR browsers** show points of interest using GPS + camera; tourism, exploration, driving assistance
Media, advertising, gaming	Live TV overlays; sports broadcasts use AR for annotations and statistics; advertising for interactive product visualisation; gaming blends physical environment with digital gameplay
Retail	More enjoyable and interactive shopping, virtual try-on. E.g. **IKEA Place, Converse Sampler app, AR Harley-Davidson**
Architecture / interior design	Visualise designs; accurate view of what will be built. E.g. **Morpholio AR Sketchwalk**
Tourism	Immersive experience before travel; point smartphone at a landmark to learn about it in real time. E.g. **ViewRanger, AR City, World Around Me**
Education	Virtual examples and gaming elements support textbooks; immersion raises understanding and retention. E.g. **3DBear, Aurasama**
Biotechnology teaching	AR can overcome: high cost of education, location limitations, resource-intensiveness, "science is boring" perception

Medical apps and student projects *(VRAR-Introduction pg 35–41)*

- **EyeDecide:** medical app that uses the camera display to simulate the effect of specific conditions on a person's vision; doctors can show patients what their condition looks like.
- **AED app:** created by Lucien Engelen (Radboud University Nijmegen Medical Centre, Netherlands); users add places where automated external defibrillators (AEDs) are located; database accessible through the app.
- **Project – AR Classroom:** pointing the camera at a picture of a human anatomical structure generates a **3D model** of it, demonstrated with visual and sound effects.

- **Project – IC engine using AR:** detailed information on internal combustion engines (fuel ignited inside the engine to do work; used in vehicles, boats, ships, aircraft, trains).

> **Figure refs:** AR heart-book on a phone – `VRAR-Introduction_PPT_1__pg_9` ; "Applications of AR" – `_pg_22` ; AR headset in surgery – `_pg_24` ; industry/factory – `ARandApplications_pg_9` ; maintenance – `ARandApplications_pg_10` ; medical – `ARandApplications_pg_11` ; AR navigation display – `ARandApplications_pg_12` ; cricket broadcast overlay – `ARandApplications_pg_13` .

4.3 Applications of MR *(PPT-Mixed_Reality pg 8, 10)*

Application	Description
Digital twins	Real-time 3D replicas of physical systems; overlay live **IoT sensor data** on the model → performance insights, failure identification, maintenance scheduling, design validation, virtual testing, **predictive maintenance**
Remote collaboration	Shared holographic workspaces; engineers in different locations interact with 3D models as if in the same room; synchronised spatial annotations (markups, measurements, pointers)
Training & simulation	Immersive, risk-free skill development (machinery, procedures, hazardous conditions); performance analytics
Material science & fabrication	Visualise CAD models in 3D, real-time finite element analysis, overlay stress/strain/thermal data on virtual prototypes before fabrication

Industry transformation (pg 10): manufacturing (design reviews, remote assistance), healthcare (surgical training, telemedicine), education (virtual classrooms), entertainment (gaming, live events), defense (training simulations, mission planning). The deck projects MR market growth to $300 billion by 2030 (a projection stated in the slides without a source).

5. Hardware

Sources: Hardware_and_software_VRAR pg 6–17, UNIT1 pg 8–18, virtual_reality_notes pg 6–10, 19–22, PPT-Mixed_Reality pg 5–6

5.1 VR hardware – overview *(Hardware_and_software pg 6–12)*

VR hardware = the physical devices needed to experience VR. It **displays** the virtual world, **tracks** movements and **enables interaction**. Components: (1) Head-Mounted Display, (2) input devices & controllers, (3) tracking sensors, (4) audio devices, (5) VR-ready computer/console.

Component	Details
HMD	Worn like goggles; **two small displays** (one per eye) → stereoscopic 3D; tracks head movement (left, right, up, down); 360° view. Examples: Oculus Quest, HTC Vive
Input devices & controllers	Handheld controllers, data gloves, motion controllers; detect hand movement, button presses, gestures
Tracking sensors	Track position and orientation; **gyroscopes, accelerometers, infrared cameras**
Audio devices	3D spatial sound; help locate objects by sound; headphones, built-in headset speakers
VR-ready computer / console	High-speed CPU, powerful GPU, large RAM. Types: PC-based, console-based, standalone headsets

5.2 Head-Mounted Display in detail *(virtual_reality_notes pg 6–7)*

- The HMD was the **first device** to give an immersive experience. Evans & Sutherland demonstrated a head-mounted stereo display in **1965**; VPL Research's commercial "**EyePhone**" came in **1989**.
- Designed to exploit **binocular vision**. Characteristics: headgear with **two small LCD colour screens**, each optically channelled to one eye; **special optics** for wide field of view; a **tracking system** for precise head location.
- A **motion tracker** continuously measures head position and orientation, so the image-generating computer updates the scene → the viewer can look around and walk through the VE.
- HMDs are intrusive/uncomfortable → alternatives **BOOM** and **CAVE**.

> **Figure ref:** HMD – `virtual_reality_notes_docx-1_pg_6` (Fig 3.1).

5.3 BOOM and CAVE *(pg 7–8)*

	BOOM	CAVE
Full form / maker	Binocular Omni-Orientation Monitor (Fakespace)	Developed at University of Illinois; sold by Pyramid Systems Inc.
Type	Head-**coupled** stereoscopic display	**Multi-person**, room-sized, high-resolution 3D video and audio environment
How it works	Screens + optics in a **box on a multi-link arm**; user looks through two holes and moves the box anywhere in its operating volume. Head tracking by **sensors in the arm's links**	**Four projectors** throw images on **three walls and the floor** (software could support 6 walls); software synchronises devices and computes perspective for each wall from the user's viewpoint; head tracker gives position; **offset images for each eye**

> **Figure refs:** BOOM – `virtual_reality_notes_docx-1_pg_7` (Fig 3.2); CAVE schematic – `virtual_reality_notes_docx-1_pg_8` (Fig 3.3).

5.4 Input devices *(virtual_reality_notes pg 8–10)*

Data glove (VPL DataGlove)

- Hand-measurement devices must sense **finger flex angles** and the **position/orientation of the wrist** in real time. First commercial one = VPL Research's **Data Glove**.
- Lightweight nylon glove with **optical sensors along the fingers**. Each sensor = a short **fibre-optic cable** with an **LED** at one end and a **phototransistor** at the other. When the cable bends, light is lost → less light received → bend angle.
- Measures bending of the thumb joints and the lower and middle knuckles of the other fingers (ten flex angles); can be extended to measure abduction between fingers.
- A **Polhemus sensor** on the back of the hand measures position and orientation; data goes via a **serial line** to the host computer.

3D mouse and SpaceBall

- **Logitech 3D mouse:** based on an **ultrasonic position reference array** – a tripod of three ultrasonic speakers in a triangle emits signals used to track the receiver's position, orientation and movement. Gives output in all **6 DOF**: X, Y, Z, Pitch, Yaw, Roll.
- **SpaceBall** (Spatial Systems): 6-DOF **force-sensitive** device; force and torque applied to the ball are sent in real time to the computer, interpreted and composed into homogeneous transformation matrices applied to objects.

MIDI keyboard

Originally for music, but a general way to enter multi-dimensional data simultaneously; good for controlling many DOFs in real-time animation. 88 keys, each transmits **velocity** of keystroke and **pressure** after the key is pressed.

> **Figure refs:** data glove – `virtual_reality_notes_docx-1_pg_9` (Fig 3.4); 3D mouse/ball – `virtual_reality_notes_docx-1_pg_10` .

5.5 Coordinate system and trackers *(UNIT1 pg 9–14)*

Coordinates of a moving 3D object

Position along X, Y, Z plus rotation angles **roll, pitch, yaw** (six degrees of freedom). In the slide's diagram: Z-axis ↔ roll, Y-axis ↔ pitch, X-axis ↔ yaw. Figure: `UNIT1_pg_9` .

> **Tracker:** special-purpose hardware used in VR to measure the **real-time change in a 3D object's position and orientation**. *(UNIT1 pg 11)*

Tracker parameter	Meaning
Accuracy	Difference between the object's **actual 3D position** and that reported by the tracker (how precisely movement is captured)
Jitter	Change in tracker output when the tracked object is **stationary** – small, rapid, unwanted fluctuations ("noise") causing instability
Drift	**Steady increase in tracking error with time** – virtual environment slowly shifts from the user's actual position; causes discomfort, disorientation
Latency	**Time delay** between the change in object position and the time the sensor detects/reports it (delay between physical movement and update in the virtual environment)

> **Figure refs:** tracker performance parameters (a) accuracy (with resolution and repeatability), (b) jitter, (c) drift, (d) latency – `UNIT1_pg_13` (Fig 2.3); 3D-sound/tracker setup with LCD screens, optics, speaker, receiver, transmitter, left-/right-eye video generators, 3D sound generator and position/orientation tracker – `UNIT1_pg_10`.

Mechanical trackers *(pg 14)*

- The **first tracker**: a mechanical arm that supported a **CRT-based HMD**; head motion tracked relative to the ceiling-arm attachment.
- Accuracy fairly constant; **very low jitter and the lowest latency**. Used as part of the "Push" display.
- **Drawbacks:** limited range, reduced freedom of motion.
- Graph on the slide: **Tracker A** keeps 144 updates/sec regardless of the number of receivers; **Tracker B** falls from 120 → 60 → 30 updates/sec as receivers go 1 → 2 → 4 (sampling rate drops with more receivers). Figure: `UNIT1_pg_14`.

CRT-based HMDs *(pg 15)*

Use **Cathode Ray Tube** technology; used in early VR. **Pros:** high resolution, good colour. **Cons:** bulky and heavy, **screen flicker** (eye strain), lower refresh rates (motion blur).

5.6 Navigation & manipulation interfaces *(UNIT1 pg 16)*

> **Navigation/manipulation interface:** a device that allows interactive change of the view of the virtual environment and exploration through selection and manipulation of a virtual object of interest.

Absolute sensors (trackers)	Relative sensors
Return position/orientation w.r.t. a **fixed** coordinate system	Return **zeros** if not acted upon
Data is **never a zero set**, even if the receiver is at rest	Another way to control a VR object's position

5.7 Gesture interfaces *(UNIT1 pg 17–18)*

- **Pinch Glove** (block diagram `UNIT1_pg_17`): Lycra glove with **conductive fabric** and **fingertip electrodes**; a multiplexer chip and signal/power lines go through a glove connector to **interface electronics**, which talks over **RS232** to the host computer. Gestures are detected when fingertip contacts close the circuit (thumb, index, middle, ring, pinkie).
- **5DT Data Glove** (`UNIT1_pg_18`): **fibre-optic sensors** on fingers + opto-electronics + a **tilt sensor**; interface box and cable to a **wireless transmitter** → **wireless receiver** → **serial connector (DB9)** on the host computer; powered by a 9 V DC adapter.

5.8 Consumer devices *(virtual_reality_notes pg 19–22)*

Google Cardboard	Oculus Rift
• VR/AR platform by Google using a **head mount for a mobile phone**; named for its fold-out cardboard viewer. • **Low-cost** system to encourage VR/AR interest and development; build your own from published specs or buy one. • Created by David Coz and Damien Henry (Google Cultural Institute, Paris, "20% time"); introduced at Google I/O **2014** (Android), iOS support 2015. • By Jan 2016: >5 million viewers shipped, >1,000 apps. • Two SDKs (both use **OpenGL**): Android (Java) and **Unity** (C#); also Chrome VR Experiments via **WebGL**; SDKs support **spatial audio**.	• VR headset by **Oculus VR**; funded by Kickstarter (US$2.5 million). • Release scheduled **28 March 2016**; among first consumer-targeted VR headsets. • **1080×1200 per eye, 90 Hz** refresh rate, wide field of view, integrated headphones giving **3D audio**. • **Rotational + positional tracking**; positional tracking by a **USB stationary IR sensor** on the desk (use sitting, standing or walking in the same room). • Development kits **DK1** (late 2012) and **DK2** (mid 2014).

> **Figure refs:** `virtual_reality_notes_docx-1_pg_20` (Fig 6.1 Google Cardboard); `virtual_reality_notes_docx-1_pg_22` (Fig 6.2 Oculus Rift).

5.9 AR hardware *(Hardware_and_software pg 13–17)*

AR hardware lets digital content be overlaid on the real world; it does **not block** the real environment; widely used in mobile apps. Components: display devices, cameras, sensors, processing unit.

- **Display devices:** smartphones, tablets, AR (smart) glasses.
- **Cameras:** capture the real world; detect surfaces, objects, markers; essential for placing virtual objects accurately.
- **Sensors:** gyroscope, accelerometer, GPS → location tracking, orientation detection, movement detection.
- See also AR device types (HUD, holographic, smart glasses, handheld) in section 2.5.

5.10 MR sensing hardware *(PPT-Mixed_Reality pg 3–6)*

RGB cameras, depth sensors, IMUs (sensor fusion); LIDAR, stereo cameras, time-of-flight sensors; eye-tracking and hand-tracking sensors; Ultraleap (infrared hand tracking + mid-air haptics) and Kinect (full-body). Details in section 3.

6. Software

Sources: Hardware_and_software_VRAR pg 18–22, UNIT1 pg 6, VRAR-Introduction pg 11, ARandApplications pg 8, PPT-Mixed_Reality pg 4–7, virtual_reality_notes pg 13–14, 20

> **VR/AR software:** creates and controls virtual environments (VR) and augmented objects (AR) and defines **object behaviour, interaction rules and movement logic**. *(Hardware_and_software pg 18)*

Type	Role	Examples
System software	Backbone that runs the VR/AR hardware and manages resources; controls hardware devices, manages sensors and displays, **bridge between hardware and applications**. Without it the hardware cannot function	Device drivers, OS support libraries, device operating systems (VR headsets, AR glasses), hardware drivers (motion sensors, cameras, controllers)
Application software	What end users interact with; education, entertainment, training	VR games, AR navigation apps, medical simulators
Development software	Used by programmers, designers, developers to build VR/AR apps; provides tools for **3D modelling, animation, physics**	Unity, Unreal Engine

Specific software / SDKs mentioned

- **Unity** – most widely used VR/AR/MR development platform (notes: >91% of Vuforia and HoloLens experiences). **Vuforia Engine** – AR app platform with computer vision (recognise images and objects).
- **ARToolKit** – released **1999** as the **first open-source AR platform**.

- **MRTK** – open-source cross-platform Unity SDK for MR (HoloLens 2, Meta Quest, Windows MR, OpenXR devices). **Scene Understanding SDK** – plane detection, object recognition, semantic labelling.
- **Google Cardboard SDKs** – Android (Java) and Unity (C#), both OpenGL based; **WebGL** for browser experiments.
- **VRML / VRML97** – language for 3D worlds on the web.
- Rendering and compute: GPU-accelerated rendering, cloud and edge computing (see MR, section 3.3).
- Requirements from the notes' conclusion: high-performance computers, associated software and **high-bandwidth networks** to keep pace with real-time interaction. *(virtual_reality_notes pg 23–24)*

7. Human physiology & perception

Source: Human_Physiology_and_perception_of_VRAR (pg 2–15)

7.1 Why physiology matters *(pg 2–3)*

- VR/AR systems interact **directly with human sensory organs**; perception determines how realistic the experience feels. The brain interprets VR/AR stimuli as real.
- **Sensory conflicts** cause discomfort and nausea. Physiological limitations define system design constraints; user experience depends on sensory accuracy; poor design reduces usability and adoption.

7.2 Human sensory systems involved *(pg 4)*

System	Function
Visual	Processes images, depth, motion
Auditory	Processes sound direction and distance
Vestibular	Maintains balance and orientation
Proprioceptive	Tracks body position
Tactile	Processes touch and pressure

> **Figure ref:** `Human_Physiology_and_perception_of_VRAR_pg_4` – brain at centre with the five sensory systems.

7.3 Visual perception *(pg 5–7)*

- **Vision is the dominant sense** in VR/AR; humans rely heavily on visual cues for realism.
- **Resolution** → image clarity; **frame rate** → smoothness; **latency** → eye–brain coordination.

Binocular vision and depth perception *(pg 6)*

Two eyes separated by a small distance; **each eye sees a slightly different image**; the brain combines them to perceive depth. VR uses **stereoscopic displays** to simulate depth; **incorrect depth cues cause eye strain**.

Field of View (FOV) ***(pg 7)***

FOV = visible area seen by the eyes; human horizontal FOV ≈ **200°**. Narrow FOV reduces immersion; wide FOV increases realism but may cause discomfort; headsets aim for a **balanced FOV**.

7.4 Motion perception, vestibular system and sensory conflict *(pg 8–10)*

- Humans are sensitive to visual motion. VR simulates movement artificially; **visual motion without physical movement confuses the brain**; sudden acceleration worsens discomfort; smooth, predictable motion improves comfort.
- **Vestibular system:** in the **inner ear**; detects head movement and acceleration; maintains balance and spatial orientation; works with vision. Conflict with visual cues → **motion sickness**.

> **Sensory conflict theory:** occurs when senses give conflicting information – the eyes perceive motion but the body is stationary; the brain cannot resolve the mismatch → nausea and dizziness. Common in VR.

> **Exam tip:** "Why does VR cause motion sickness?" → eyes say "moving", vestibular system (inner ear) says "stationary" → sensory conflict → nausea/dizziness. Add: latency and sudden acceleration make it worse.

7.5 Auditory, proprioception and haptics *(pg 11–13)*

Sense	Key points
Auditory (pg 11)	Hearing is important for immersion; humans locate sound using **binaural hearing**; **spatial audio** creates 3D sound effects; directional sound improves realism; audio delay or mismatch reduces immersion
Proprioception (pg 12)	Sense of body position – know where limbs are without seeing them; VR controllers track hand and body movement; accurate tracking improves realism; mismatch causes confusion and discomfort
Haptic feedback (pg 13)	Provides a sense of touch using **vibration and force-feedback** devices; enhances VR training systems; common in controllers and gloves; limited compared with real-world touch

7.6 Perception challenges in AR *(pg 14)*

- AR overlays digital objects on the real world → **proper alignment is critical**; depth and occlusion errors are easily noticed; lighting mismatch reduces realism; human perception quickly detects inconsistencies.
- Figure: `Human_Physiology_and_perception_of_VRAR_pg_14` (virtual mug showing a "tracking error").

Conclusion *(pg 15)*: perception is central; sensory systems must work together; mismatches cause discomfort and sickness; physiology-based design improves usability.

8. History of VR and AR

Sources: virtual_reality_notes pg 1–5, 19–22; ARandApplications pg 7–8; VRAR-Introduction pg 10

Year	Event
Mid 1950s	**Morton Heilig** (cinematographer) envisioned an "Experience Theater" that would engage all the senses
1960 / 1962	Heilig's **Sensorama** – single-user console with stereoscopic display, fans, odour emitters, stereo speakers, moving chair. The notes give 1960 (pg 3) and 1962 for the prototype with five short films (pg 5). It was mechanical (predates digital computing); users were passive. He also invented a head-mounted TV display
1961	**Philco Corporation** engineers built the first HMD – **Headsight** (video screen + tracking linked to a closed-circuit camera, for dangerous situations; turning the head moved the camera). Bell Laboratories used a similar HMD with infrared cameras for helicopter pilots flying in the dark
1960s	AR: **Ivan Sutherland** proposed interactive computer displays
1965	Evans & Sutherland demonstrated a head-mounted stereo display
1968	Ivan Sutherland (with student Bob Sproull) built what is widely considered the **first VR and AR HMD** – the **"Sword of Damocles"**: so heavy it hung from the ceiling, simple wireframe room graphics; included **head tracking and see-through optics**
1970s	"Artificial reality" term by Myron Krueger; interactive video overlays explored (1970s–80s) in research and art
1977	**Aspen Movie Map** (MIT) – crude virtual simulation of Aspen; modes: summer, winter, polygons
1982–84	"Virtual reality" credited to *The Judas Mandala* (1982 novel, Damien Broderick); "cyberspace" from Gibson's *Burning Chrome* (1982) and *Neuromancer* (1984)
1985	**Jaron Lanier** founded **VPL Research** ("goggles n' gloves" systems); he popularised the term "virtual reality" in the late 1980s
1989	VPL's commercial HMD – the **EyePhone**
1990s	Public awareness of VR; research boom (motivated by Howard Rheingold's book *Virtual Reality*); films like *The Lawnmower Man*; handheld and collaborative AR systems developed
1992	Term **"Augmented Reality"** introduced in an industrial context (assembly and maintenance guidance); earliest functional AR system – **Virtual Fixtures**, U.S. Air Force Armstrong Laboratory
1993	VR art exhibit at the Soho Guggenheim Museum
1999	**ARToolKit** – first open-source AR platform; *The Matrix* released
2012 / 2014	Oculus Rift DK1 (late 2012), DK2 (mid 2014); Google Cardboard introduced at Google I/O 2014
2016	Oculus Rift consumer release scheduled 28 March 2016

Later	Advances in mobile computing enabled modern AR systems; commercial AR started in entertainment/gaming, then education, communications, medicine

> **Figure refs:** Sword of Damocles HMD photo – `ARandApplications_pg_7` ; early AR headset user (1992 era) – `ARandApplications_pg_8` .

> **Inconsistency in the notes:** Sensorama date is 1960 on one page and 1962 on another; Sutherland's HMD is 1965 (Evans & Sutherland) in the HMD section and 1968 in the timeline. In an exam, state 1968 for the Sword of Damocles and mention Sensorama as ~1960s.

Unit I – rapid revision Q&A

Define VR (formal).

> Inducing targeted behavior in an organism using artificial sensory stimulation, with little or no awareness of the interference.

List the four key elements of VR.

> Targeted behavior, artificial sensory stimulation, organism, unawareness.

Three I's of VR?

> Immersion, Interaction, Imagination.

Five classic components of a VR system?

> VR engine, I/O devices, software & databases, user, task.

Open-loop vs closed-loop?

> Open-loop: no response to user actions (360° video). Closed-loop: output updated from user feedback (VR games, simulators).

Azuma's three AR requirements?

> Combine real and virtual, interactive in real time, registered in 3D.

Four AR device types?

> HUD, holographic displays, smart glasses, handheld.

AR tracking techniques?

> Sensor-based (optical, magnetic, acoustic, inertial) and vision-based.

VR vs AR vs MR?

> VR replaces the real world; AR overlays digital info on it; MR lets digital content interact dynamically with the physical environment.

What is the virtuality continuum?

> Milgram & Kishino 1994: pure reality ↔ pure virtuality; MR (AR + AV) lies between.

Tracker parameters?

> Accuracy, jitter, drift, latency (definitions in 5.5).

Difference between HMD, BOOM and CAVE?

> HMD worn on head (two LCDs + optics + tracker); BOOM is a head-coupled box on a multi-link arm; CAVE is a room-sized multi-person projection environment.

How does a DataGlove sense finger bending?

> Fibre-optic cable with LED and phototransistor – bending loses light; Polhemus sensor gives hand position/orientation.

Why does VR cause motion sickness?

> Sensory conflict between vision and the vestibular system.

Types of VR/AR software?

> System, application, development software (Unity, Unreal).

Who built the Sword of Damocles and why that name?

> Ivan Sutherland (1968); so heavy it was suspended from the ceiling – intimidating appearance.

UNIT II – Programming with Unity

Syllabus: Unity Basics · Manipulating the Scene · Code blocks and Methods · Debugging Conditional and looping statements · Working with objects · Working with scripts

1. Unity basics

Source: UnityBasics-ManipulatingScenes_PPT_1__pptx (pg 2–12)

1.1 Unity windows and layouts *(pg 2–3)*

The Unity Editor is divided into windows, each with a specific task.

Window	Purpose
Project	Shows **all your assets** (art, sound, code, levels)
Scene	View and navigate the game environment
Hierarchy	Lists **all objects** in the current scene
Inspector	View and edit **properties** of the selected object

- **Window management:** drag and drop to reposition, resize by dragging edges, dock side by side with tabs, remove windows you don't need; each window has a tab at its top-left.
- **Layouts:** window arrangements can be saved; the **Layout** button is at the **top-right**. Built-in layouts: **Default, 2 by 3, Tall, Wide**, or create your own ("Save Layout..."). Different layouts suit coding, scene design, debugging.

1.2 Project window *(pg 4)*

- An **asset** = any piece of work used in a game: art, sound effects, music, code files, game levels.
- Works like a computer **file system** – assets stored in folders (e.g. sound, scripts, art/models).
- Default folders: **Packages** and **Assets**. Assets by default contains a **Scenes** folder with a scene asset named **SampleScene**.

1.3 Scene window *(pg 5–6)*

- A game **level** is called a **scene**. Scenes are saved as assets and appear in the Project window; default = SampleScene. Each scene has its own collection of objects.
- The Scene window is the main **viewport** – like a **floating camera** observing the game world.

- SampleScene initially contains a **light** (invisible object casting light) and a **camera** (the object through which the player sees the scene during gameplay).
- Double-click a scene to load it (to view/edit).
- **Navigation:** hold **right mouse button** and move the mouse to turn the camera; while holding it: **W** forward, **S** backward, **A** left, **D** right, **Q** down, **E** up.

1.4 Hierarchy window *(pg 7)*

- Shows the objects in the current scene. A scene is a collection of objects; **switching scenes** removes the current scene's objects and loads the new scene's objects.
- Default scene lists **Directional Light** and **Main Camera** – called **GameObjects**.

> **GameObject:** any object in a scene – props (box, plant, tree), player character, enemy, powerup, light, or an invisible object. At the simplest level, a point in space with a name.

1.5 Inspector window *(pg 8)*

- Click a GameObject in the Hierarchy to select it → the Inspector changes to show information about it.
- At the top: the GameObject **name** (can be renamed) and dropdowns for **Tag** and **Layer**.
- Main function: display **all components attached** to the selected GameObject(s).

1.6 Components *(pg 9–10)*

> **Component:** a single piece of game functionality attached to a GameObject. A component **cannot exist without a GameObject**.

- Built-in components for many purposes, e.g. the **Light** component (sun-like light, flashlight-style light). A GameObject can have multiple components, e.g. Transform and Light.
- Properties are edited with value fields (numbers, sliders) in the Inspector.
- The **Camera** component renders the scene to the player's screen during gameplay.
- **User-written code** is added as **script components** attached to GameObjects – reusable and mixable.
- **Combining components:** behaviours like Shooting, Spinning, Patrolling, Hazard can be separate components; attach several to one GameObject. Unity's component system is a **building-block system** that allows flexible feature combinations.

1.7 Adding GameObjects *(pg 11–12)*

- Menu bar: File, Edit, Assets, GameObject, Component, Window, Help.
- Create a cube: `GameObject ➤ 3D Object ➤ Cube`, or right-click in the Hierarchy.
- The cube is added to the Hierarchy; appears in the Scene if the camera is positioned correctly; press **F** to focus the Scene view on the selected object.

- Every GameObject has a **Transform** component (position, rotation, scale) – cannot be removed without deleting the GameObject.
- A cube also has **Mesh Filter** (holds the 3D model) and **Mesh Renderer** (displays the model on screen; can be enabled/disabled via its checkbox).

> **Figure refs (screenshots):** editor windows – `UnityBasics-ManipulatingScenes_PPT_1__pptx_pg_2` ; Project window `_pg_4` ; Scene window `_pg_5` ; Hierarchy `_pg_7` ; Inspector `_pg_8` ; light component `_pg_9` ; GameObject menu / cube `_pg_11` , `_pg_12` .

2. Manipulating the scene

Source: UnityBasics-ManipulatingScenes_PPT_1__pptx (pg 13–20)

2.1 Transform tools *(pg 13–15)*

- The **Toolbar** sits just below the title bar and spans the screen width. Its leftmost buttons are the **Transform Tools**, used to manipulate GameObjects; they match the Transform component (**Position, Rotation, Scale**). There are **six** tools; only **one is active** at a time.

Tool	Key	What it does
Hand	**Q**	Moves the **Scene camera** only; does not modify GameObjects; helps navigation
Move (Position)	**W**	Arrow gizmos; drag an arrow to move along X, Y or Z; drag the square areas to move along two axes at once; hold **Ctrl** for 1-unit increments
Rotate	**E**	Circular gizmos; drag circles to rotate around different axes; drag between circles to rotate about several axes
Scale	**R**	Cube-ended arrows; scale per axis – **Red → width (X), Blue → length (Z), Green → height (Y)**; drag the centre cube to scale **uniformly**
Rect	**T**	Mainly for **2D**; rectangle gizmo with corner handles; adjusts position and scale together; resize on one side only; gizmo auto-faces the camera
Transform	**Y**	Combines Move, Rotate and Scale (arrows + rotation circles + scale cube together) for quick editing

> **Mnemonic:** Q W E R T Y = Hand, Move, Rotate, Scale, Rect, Transform. Note that Q/E also mean "down/up" for camera flight while the right mouse button is held (pg 6) – different context.

2.2 Positions and axes *(pg 16–17)*

- 3D position = three values **(X, Y, Z)**, e.g. (15, 20, 25). **World origin = (0, 0, 0)**, the centre of the scene.

- X: left (−) ↔ right (+). Y: down (−) ↔ up (+). Z: backward (−) ↔ forward (+). Adding a value moves in the positive direction; subtracting moves negative.
- The same axes are used for **position, scale and rotation**. Scale: X = width, Y = height, Z = length. Rotation: three angles (0°–360°), one per axis.
- **Colour coding:** X = **red**, Y = **green**, Z = **blue** (some 3D software swaps Y and Z directions; Unity follows the standard above).

2.3 Making a floor *(pg 18)*

1. `GameObject → 3D Object → Plane`. A **Plane** is a thin flat surface with **no thickness**, **one-sided** (invisible from below) – fine for a floor.
2. Transform = Position (where), Rotation (orientation), Scale (size).
3. Select the Plane → Inspector → Transform → set Position to **(0, 0, 0)** (world origin) and keep Rotation at (0, 0, 0).
4. **Why the Inspector?** exact numeric control; faster and more accurate than gizmos.

2.4 Scale and unit measurements *(pg 19–20)*

- Unity does **not enforce real-world units** – a "unit" is a design decision. E.g. if 1 unit = 1 foot: human height ≈ 5–6 units, 1 inch ≈ 0.083 units, 1 yard ≈ 3 units.
- **Consistency matters more than the unit itself.**
- **Scale ≠ size:** scale is a **multiplier**, not actual size; every object's mesh has its own dimensions. So a Plane looks much larger than a Cube even with the same scale (1, 1, 1).

> **Figure refs:** transform tools toolbar – `UnityBasics-ManipulatingScenes_PPT_1__pptx_pg_13` to `_pg_15`; axes/colour gizmos – `_pg_16`, `_pg_17`; plane and cube comparison – `_pg_20`.

3. Code blocks and methods (C# basics)

Source: CODE_BLOCKS_-_METHODS_PPT_pptx (pg 3–20)

3.1 Statements and semicolons *(pg 4–5)*

> **Statement:** a single instruction given to the computer. Every C# statement must end with a semicolon `;`.

```
int score = 10;
Debug.Log(score);
```

- The semicolon tells the compiler **"this instruction is complete"**. Missing it → the program will not run; the compiler shows an error (very common beginner mistake).

- Wrong: `int x = 5` · Right: `int x = 5;`

3.2 Code blocks *(pg 6–8)*

> **Code block:** a group of statements written inside curly braces `{ }`; all statements inside run together.

```
{
    int a = 5;
    int b = 10;
}
```

- Needed to organise code and keep it readable. Used when writing **methods**, **conditions (if)** and **loops**.
- `{` = start of block, `}` = end of block.

```
if (score > 50)
{
    Debug.Log("Pass");
}
```

3.3 Comments *(pg 9–11)*

- Notes written in code that the computer **ignores**; explain what the code does.
- **Single-line:** `// This prints the score` · **Multi-line:** `/* This method calculates score */`
- Importance: help beginners understand code, easy to revise later, useful for teachers/teamwork, help during debugging. "Good code = code + comments".

3.4 Data types *(pg 15)*

Type	Stores	Example
`int`	Whole numbers	10, 25
`float`	Decimal numbers	3.5
`bool`	true / false	true
`string`	Text	"Hello"

3.5 Methods *(pg 12–14, 16–18)*

> **Method:** a block of code that performs a task; helps reuse code; runs **only when called**.

```
void SayHello()
{
    Debug.Log("Hello");
}
SayHello();   // calling the method – runs the code inside
```

- **Advantages:** avoid repeating code, shorter programs, better readability, easier to fix errors. "One method → used many times."
- A method does nothing until **called** (method name followed by parentheses).

Methods with parameters *(pg 16)*

Parameters let a method receive values, passed when it is called.

```
void ShowNumber(int num)
{
    Debug.Log(num);
}
ShowNumber(5);
```

Methods that return values *(pg 17)*

Use the `return` keyword to send a result back.

```
int Add(int a, int b)
{
    return a + b;
}
int sum = Add(2, 3);
```

Method structure *(pg 18)*

```
returnType MethodName(parameters)
{
    statements;
}

void Jump()
{
    Debug.Log("Jumping");
}
```

3.6 Common beginner mistakes *(pg 19)*

Forgetting the semicolon · missing curly braces · **not calling methods** · wrong data type usage · writing code **outside methods**.

Summary *(pg 20)*: statements end with `;` ; code blocks use `{ }` ; comments explain code; methods organise tasks and can return values; clean code is easy to understand.

4. Debugging conditional and looping statements

Source: Debugging_Conditional_-_Looping_Statements_pptx (pg 2–13)

4.1 Why debugging is critical in AR/VR *(pg 2–4)*

> **Debugging:** the process of identifying and fixing errors in a program. Errors may be **logical, runtime or performance-related**.

- **Real-time execution:** AR/VR logic runs **every frame**, typically **60–120 frames per second** – no tolerance for inefficient or incorrect code.
- **High sensitivity to errors:** even minor logic mistakes are immediately visible because the environment responds continuously to user movement and input.
- **Object visibility issues** (wrong conditionals): objects don't appear when required, disappear suddenly, flicker or behave inconsistently.
- **Infinite loop risks:** freeze the application; force users to restart the headset.
- **Interaction failures:** wrong conditions for controller triggers, hand tracking or gaze detection → actions don't execute even when the user does the right thing.
- **Performance degradation:** poorly optimised loops running continuously use excess CPU/GPU → frame drops, lag, stuttering visuals.
- **Motion-sickness concerns:** inconsistent frame rates and delayed responses cause disorientation, eye strain, motion sickness.

4.2 Role of program logic in AR/VR *(pg 5–6)*

Program logic is the decision-making core: it determines **what happens, when and why** in the virtual environment.

Function	What the logic does
1. User input handling	Captures controller, gesture, gaze, voice, touch input. Checks: which button is pressed? how long? is the user pointing at a valid object? Responses: grab, teleport, select
2. Object interaction control	How objects react to the user and environment (touch → highlight; grabbed → disable gravity); prevents invalid actions (grabbing locked objects)
3. Animation flow management	When animations start, stop, loop or transition (walking animation only when the user moves; stop when interaction ends); smooth, no abrupt transitions
4. Scene transitions	Movement between scenes/levels based on: task completed? timer expired? correct interaction performed?
5. Backbone: conditionals & loops	**Conditionals** (if–else, switch) decide based on user action/system state. **Loops** (for, while) run continuous per-frame updates (game loop): check collisions, track user position, update animations. Together they keep real-time responsiveness at 60–120 FPS

> **Figure ref:** Input → Processing → Output diagram (body motion, controller, hand motion, voice, location → processing → vision, touch, audio, smell, taste) – `Debugging_Conditional_-_Looping_Statements_pptx_pg_5` .

4.3 Conditional statements *(pg 7–8)*

> **Conditional statements:** programming constructs for **decision making**; the program chooses different actions based on a condition that always evaluates to **true or false**.

Statement	Use
`if`	Executes code when a condition is true
`if-else`	Chooses between two paths
`else if`	Checks multiple conditions sequentially
`switch`	Selects an execution path based on fixed values

Uses in AR/VR *(pg 8)*: detect controller button presses; check gaze-based interaction (trigger event when the user looks at an object); enable/disable virtual objects based on progress; verify AR tracking status (ensure camera and surface tracking are active before placing objects); switch menus and scenes (load the right scene).

> **Figure ref:** flowchart START → CONDITION → TRUE: Statement 1 / FALSE: Statement 2 → END – `Debugging_Conditional_-_Looping_Statements_pptx_pg_7` .

4.4 Common errors in conditionals and their effects *(pg 9)*

Common errors

- Assignment `=` instead of comparison `==`
- Incorrect logical operators (`&&` , `||` , `!`)
- Condition value **never updates** at runtime
- Conditions placed **outside the update loop**
- **Wrong variable** used in the check

Effects in AR/VR

- Controller buttons do not respond
- Objects behave unexpectedly/incorrectly
- Scenes change without user intent
- Interactions feel delayed or broken
- Loss of immersion

4.5 Debugging conditional statements – best practices *(pg 10)*

- **Print variable values** using logs to track changes.
- **Verify input values at runtime** (controller, gaze, triggers).
- **Check conditions step by step.**
- **Test inside the headset** to observe real behaviour.
- **Use breakpoints** to analyse execution flow.

Common bugs & fixes *(pg 11)*

Problem	Cause	Debugging solution
Condition never executes	Wrong variable value	Print values using `Debug.Log()`
Code always runs	Missing comparison operator	Use `==` , not `=`
Wrong branch executes	Incorrect logical operators	Verify `&&` , `||` , `!`
Object not responding	Condition not updated per frame	Place logic in `Update()`

4.6 Looping statements *(pg 12–13)*

- Loops execute a block repeatedly; AR/VR is real time and needs **continuous updates** – loops refresh the environment many times per second; used for camera input, object updates, user interaction.

Loop	Behaviour	AR/VR use
`for`	Used when the number of iterations is **known**	Iterating through detected AR objects
`while`	Executes **while a condition is true**	Checking AR session or sensor readiness
`do-while`	Executes **at least once** before the condition is checked	Calibration and environment scanning
Update loop (Unity)	Runs **once every frame**	Rendering and real-time interaction

> **Performance considerations *(pg 13)*:** excessive loops cause lag and frame drops; heavy calculations reduce performance; poor optimisation can lead to motion sickness → use efficient logic inside real-time loops.

5. Working with objects (classes)

Source: objects_and_scripts_ppt_pptx (pg 2–23, 44–46) – "Game Programming with Unity and C#", Chapter 9

5.1 Programming and data *(pg 2)*

- Programming is largely about **organising and manipulating data**. Objects are single pieces of data we interact with; objects can contain other objects (a "Person" contains firstName, lastName, birthdate).
- `bool, int, float, string` are objects/types too. Goal: create our own custom objects and data types.

5.2 What is a class? *(pg 3–4)*

> **Class:** a **blueprint/template** that defines structure and behaviour; **instances** are individual copies, each with its own data (class Person → john, sarah, mike).

```
public class MyScript : MonoBehaviour {
    // Code inside class block
}
class Item { }
```

- **Access modifier** `public` = accessible from outside; `class` keyword declares a new class type; `: MonoBehaviour` = **inheritance** (makes it a Unity script component); everything between `{ }` belongs to the class; classes can be **nested**.

5.3 Variables in classes *(pg 5–7)*

```
class Item {
    string name = "Unnamed Item";
    int worth = 1;
    bool canBeSold = true;
}
```

- Syntax: `[type] [name] = [value];` Types: int, bool, float, string or custom classes.
- **Default values** if not initialised: int/float = 0, string = null, bool = false.
- **Instance variables:** a separate copy on each instance.

Instance variable	Local variable
Declared in the **class block**	Declared **inside a method**
Lives for the entire lifetime of the object	Exists only while the method runs; destroyed when it returns
Accessible by methods of the class	Scope limited to its code block; cannot be accessed from other methods

> **Figure ref:** Item class with variables inside – `objects_and_scripts_ppt_pptx_pg_6` ; class blueprint → instances – `_pg_3` .

5.4 Creating an instance and accessing members *(pg 8–9)*

```
void Start() {
    Item item = new Item();
    item.name = "Goblin Tooth";
}
```

- `new` creates a new instance in memory; `Item()` is the **constructor call**; the variable `item` stores a **reference** to the instance.
- Naming convention: **classes = PascalCase, variables = camelCase**. A **default constructor** is provided automatically if you declare none.
- **Dot operator (.)** accesses members; **assignment (=)** sets a value; types are checked (type safety).

5.5 Access modifiers: public vs private *(pg 10–11)*

`public`	`private`
Visible outside the class; **shown in the Inspector**	Hidden from outside; **not shown in the Inspector**
e.g. `public int worth;`	e.g. `private int damage;`

- **Default is private** if no modifier is written.
- **Encapsulation:** hide details, expose only what's needed. Without `public`, `item.name = "Goblin Tooth";` gives an error; adding `public` fixes it.

5.6 Debug output and string concatenation *(pg 12)*

```
Debug.Log("This " + item.name + " is worth " + item.worth + " golden coins!");
// Output: This Goblin Tooth is worth 4 golden coins!
```

`Debug.Log()` prints to the **Console** window; `+` joins strings and values; int converts automatically to string when concatenated.

5.7 Instance methods and the DRY principle *(pg 13–15)*

```
class Item {
    public string name = "Unnamed Item";
    public int worth = 1;
    public bool canBeSold = true;
    public void LogInfo() {
        if (canBeSold)
            Debug.Log("Can sell for " + worth + " coins!");
        else
            Debug.Log(name + " cannot be sold.");
    }
}
item.LogInfo();   // called through the instance
```

- **Instance methods** are declared inside the class; called via the instance; they can use the variable names directly (no `item.` prefix); they also need `public/private`; encapsulate behaviour of the object.
- Inside the method, `if (canBeSold)` needs no `== true` because it is already a bool. A single-statement if/else needs no braces.
- > **DRY – Don't Repeat Yourself:** write code once, use it many times (call `item1.LogInfo()` instead of repeating the `Debug.Log` line). If the logic changes, update one place; fewer bugs; copy-paste duplicates bugs.

5.8 Constructors and `this` *(pg 16–19)*

> **Constructor:** a special method that **initialises an object** when it is created with `new`.

```
public Item(string name, int worth, bool canBeSold) {
    this.name = name;
    this.worth = worth;
    this.canBeSold = canBeSold;
}
// Without constructor (4 lines)         // With constructor (1 line)
Item item = new Item();                  Item item = new Item("Goblin Tooth", 4, true);
item.name = "Goblin Tooth";
item.worth = 4;
item.canBeSold = true;
```

- `this` refers to the **current instance**. When a parameter and a field share a name, `this.name` = the instance field, plain `name` = the parameter. (Alternative: different names such as `_name`, but `this` is preferred.)
- Benefits: one line instead of several; argument **order must match** the constructor signature; every instance is initialised the same way; change creation logic in one place.
- **Default constructor replacement:** once you declare a custom constructor, the automatic **default constructor disappears** – `new Item()` now gives a compile error; you must call `new Item("Name", 10, true)`. Parameter count/type mismatch also gives a compiler error.

5.9 Static members *(pg 20–23)*

```
class Item {
    public static int NumberOfInstances = 0;
    public string name;
    public Item(string name) {
        NumberOfInstances += 1;   // shared counter
        this.name = name;
    }
    public static void LogInstanceCount() {
        Debug.Log("Instances: " + NumberOfInstances);
    }
}
void Start() {
    Item.LogInstanceCount();      // 0
    Item item1 = new Item("Sword");
    Item item2 = new Item("Shield");
    Item.LogInstanceCount();      // 2
}
```

Static variable	Static method
Shared by the **entire class**; all instances see the same value	Accessed through the **class name**; has no `this` (no instance context)
Access as `Item.NumberOfInstances`, not `item.NumberOfInstances`	Can access **only static members**; cannot use instance variables like `name` / `worth`
`Debug.Log()` is itself a static method of the `Debug` class.	

5.10 Practice exercise *(pg 46)*

Create an `Item` class with properties name, description, weight; a constructor initialising them; a method `DisplayInfo()` that prints to the console; make `weight` **private** and access it through a method. Steps: declare class and variables → constructor → instance method → create instances in `Start()` → test with the Console.

6. Working with scripts

Source: objects_and_scripts_ppt_pptx (pg 24–45) – Chapter 10

6.1 Scripts vs regular classes *(pg 24–25)*

Regular class	Script class
`public class Item { }`	`public class PlayerController : MonoBehaviour { }`
Instantiated in code with `new`	Instantiated in the **Editor** by attaching it to a GameObject (or at runtime with `Instantiate()`)
—	**MonoBehaviour** = Unity base class giving special features; script must be **attached to a GameObject to run**; scripts are attached like cameras/lights; public variables editable in the Inspector

6.2 `using` statements and namespaces *(pg 26)*

```
using System.Collections;           // Lists
using System.Collections.Generic;   // Generics
using UnityEngine;                  // Core Unity (Debug, Vector3, Transform ...)
```

A **namespace** is a container organising code like folders; a `using` statement gives direct access without the full path (`Debug.Log()` instead of `UnityEngine.Debug.Log()`). Common: System, UnityEngine, custom namespaces.

6.3 Script class structure *(pg 27)*

```
public class SimpleRotation : MonoBehaviour {
    // Variables at top
    public Vector3 rotationPerSecond;
    private int internalCounter = 0;
    void Start() { }   // Called once on startup
    void Update() { }  // Called every frame
}
```

- Variable declarations at the top, methods below. `public` variables appear in the Inspector (editable per instance); `private` are internal.
- `Start()` – called **once at startup**; `Update()` – called **every frame**.

6.4 File naming rule *(pg 28)*

> **Critical:** the **file name must exactly match the class name** (case-sensitive, no spaces, PascalCase, `.cs` extension). `SimpleRotation.cs` containing `public class SimpleRotation` ✓; `RotationScript.cs` containing `SimpleRotation` ✗ → script cannot be attached to a GameObject.

6.5 Inspector variable exposure *(pg 29, 43)*

```
public class Enemy : MonoBehaviour {
    public int health = 100;     // visible in Inspector
    private int damage = 10;     // hidden
    public float speed = 5.5f;   // editable per GameObject
}
```

- Each GameObject can have different values; values can be edited **while the game runs**, but they **revert to the original when play stops**.
- No initialisation → type default (0, null, false); with assignment → that value is the default; the Inspector can override it per GameObject.

6.6 Vector3, Transform and `Time.deltaTime` *(pg 30–33)*

- **Vector3** holds X, Y, Z – a position or direction in 3D space, e.g. `Vector3 pos = (5, 10, 3)`.
- `transform` is a built-in MonoBehaviour property (always available). `transform.Rotate(Vector3)` rotates the object: **X tilts forward/backward, Y spins left/right, Z rolls side to side**.

```
void Update() {
    transform.Rotate(rotationPerSecond * Time.deltaTime);
}
```

> **Time.deltaTime:** time between frames; multiplying by it makes motion **frame-rate independent**. Without it (`pos += Vector3.right * 5;`) speed depends on FPS; with it (`* Time.deltaTime`) speed is consistent.

- `public Vector3 rotationPerSecond = new Vector3(0, 45, 0);` → rotates 45° per second around the Y-axis; units are **degrees per second** (360 = full rotation); negative values rotate the opposite way; editable per axis in the Inspector, even at runtime.

6.7 Attributes *(pg 34–36)*

Attribute	Effect
`[HideInInspector]`	Hides a **public** variable from the Inspector; it stays public (other scripts can still access it)
`[Header("Text")]`	Shows a **bold section header** in the Inspector to group related variables
Stacking	Several attributes, each in its own `[ ]`, can be stacked (one per line is cleaner); common combo: Header + HideInInspector

```
[Header("Movement")]
public float speed = 5f;
[HideInInspector]
public Vector3 internalVelocity;
```

6.8 MonoBehaviour automatic members *(pg 37)*

Inheriting `: MonoBehaviour` gives automatic access to `transform` (this GameObject's Transform) and `gameObject` (this GameObject) – lowercase, never null, same GameObject. E.g. `transform.Rotate(0,45,0);` and `gameObject.SetActive(false);`.

6.9 Creating, attaching and testing SimpleRotation *(pg 38–40)*

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class SimpleRotation : MonoBehaviour {
    public Vector3 rotationPerSecond;
    void Update() {
        transform.Rotate(rotationPerSecond * Time.deltaTime);
    }
}
```

1. **Create** the script in the Project window; **save** it in the code editor.
2. **Attach:** (a) drag the `.cs` file from Project onto the GameObject in the Hierarchy, or (b) select the GameObject → Inspector → **Add Component** → SimpleRotation.
3. The public `Vector3` shows **three input fields** (X, Y, Z). Set Y = 45.
4. **Press Play** → object rotates 45°/second around Y. Values can be changed live; they reset on Stop.

6.10 Transform common methods *(pg 41)*

Purpose	Code on the slide
Position – set exact location	`transform.position = new Vector3(x, y, z);`
Rotation – spin around axes	`transform.Rotate(45, 0, 0);`
Scale – change size	`transform.scale = new Vector3(2, 2, 2);`

> **Errors on this slide (pg 41):** the rotation line is printed as `45Q` (typo for `45`), and the property for size in Unity is `transform.localScale`, not `transform.scale`. Write `localScale` in exams, but know what the slide shows.

6.11 Best practices and common mistakes *(pg 44–45)*

Best practices

- Naming: classes = PascalCase, variables = camelCase, **file name = class name**
- Variables at top, methods below, logical grouping
- **Encapsulation:** private by default, public only if needed
- **DRY:** extract repeated code into methods/classes
- Comment complex logic
- Use Headers to organise the Inspector

Common mistakes

- Forgetting `public` → variable won't appear in the Inspector
- Filename mismatch (`PlayerController.cs` with `class Player`) → attachment fails
- Missing `Time.deltaTime` → speed varies with frame rate
- Accessing a private member (`item.name`) → error
- Declaring a custom constructor removes the default one
- Calling a static member via an instance instead of `Class.Member`

> **Figure refs (graphics):** `objects_and_scripts_ppt_pptx_pg_10` (public vs private), `_pg_24` (scripts attached like components), `_pg_30` (Vector3 X, Y, Z), `_pg_32` (with/without deltaTime), `_pg_41` (position/rotation/scale).

Unit II – rapid revision Q&A

Name the four main Unity windows and their purpose.

> Project (assets), Scene (view/navigate the environment), Hierarchy (objects in the scene), Inspector (properties/components of the selected object).

What is a GameObject? A component?

> GameObject: any object in a scene (a named point in space). Component: a piece of functionality attached to a GameObject; it can't exist without one.

Which component does every GameObject have?

> Transform (position, rotation, scale).

Keys for the six transform tools?

> Q Hand, W Move, E Rotate, R Scale, T Rect, Y Transform.

Axis colours and directions?

> X red (left−/right+), Y green (down−/up+), Z blue (backward−/forward+); origin (0,0,0).

Scale vs size?

> Scale is a multiplier on the mesh's own dimensions; a Plane and a Cube with scale (1,1,1) look different.

What is a code block? Why use it?

> Group of statements in `{ }`; used for methods, conditions and loops; keeps code organised.

Method structure and its advantages?

> `returnType Name(parameters) { statements; }` – avoids repetition, shorter, readable, easy to fix.

Common beginner mistakes in C#?

> Missing semicolon, missing braces, not calling methods, wrong data type, code outside methods.

Why is debugging loops/conditionals critical in AR/VR?

> Logic runs every frame at 60–120 FPS; errors are instantly visible; infinite loops freeze the headset; slow loops cause frame drops and motion sickness.

Typical conditional bugs and fixes?

> `=` vs `==`; wrong logical operators; value not updated; logic outside `Update()`; wrong variable. Fix with `Debug.Log`, breakpoints, headset testing.

for vs while vs do–while vs Update?

> for: known iterations; while: runs while condition true; do–while: runs at least once; Update: once per frame in Unity.

What is a class vs an instance?

> Class: blueprint; instance: an object created from it with `new`, holding its own data.

Instance vs local variable?

> Instance: declared in the class, lives as long as the object; local: declared in a method, destroyed when it returns.

public vs private?

> public: accessible outside and visible in the Inspector; private (default): hidden. Supports encapsulation.

What does a constructor do, and what happens to the default one?

> Initialises a new instance; declaring a custom constructor removes the default parameterless one.

Role of `this`?

> Refers to the current instance – separates a field from a same-named parameter.

Static vs instance members?

> Static belongs to the class (one shared copy, accessed by class name, no `this`); instance members belong to each object.

Script vs regular class?

> Script inherits `MonoBehaviour`, is attached to a GameObject in the Editor and runs there; regular classes use `new`.

Start() vs Update()?

> Start runs once at startup; Update runs every frame.

Why multiply by `Time.deltaTime`?

> Makes movement/rotation independent of frame rate.

Why must the file name match the class name?

> Otherwise the script cannot be attached to a GameObject.

What do [Header] and [HideInInspector] do?

> Header adds a bold group title in the Inspector; HideInInspector hides a public variable from it.

End of notes. All content is taken from your uploaded PDFs; the three red "inconsistency/error" boxes point out places where the notes contradict themselves.